Compiler-Guided Repair for Natural-Language-to-Lean Formalization

Author metadata pending

Affiliation and correspondence metadata pending

Abstract. Does compiler diagnostic feedback improve repair compared with equally budgeted retries, and for which errors? We describe a planned paired study of one-shot generation (A), blind retry (B), and compiler-guided repair (C), where B and C share one initial candidate and receive at most three repairs. This repository currently contains 96 dataset candidates, none eligible under the curation validator, and no qualifying paired model-run artifacts. The proposed protocol is not independently approved or preregistered. A local MLX backend is operational, but its substitution for API experiments and all mathematical study execution remain unapproved. Comparative results, uncertainty intervals, error-conditioned effects, and end-to-end faithfulness outcomes are therefore unavailable. We report the construction state, verification safeguards, and evidence gates needed before those questions can be answered. Historical manually authored compiler-checking demonstrations are identified separately and are not treated as model results.

Keywords: Lean 4 · Autoformalization · Theorem proving · Repair

1 Introduction

Natural-language-to-Lean systems have two distinct tasks: translating an informal mathematical statement into a formal proposition, and constructing a proof of that proposition. A theorem can compile while expressing a weaker, stronger, vacuous, or otherwise unintended proposition. We therefore separate repair for a fixed theorem from end-to-end faithful formalization.

This work asks: *Does compiler diagnostic feedback improve repair compared with equally budgeted retries, and for which errors?* The planned experimental unit is one eligible problem. One initial proof candidate is shared across A, B, and C. A receives no repair; B receives up to three blind repairs without diagnostics; C receives up to three repairs with real Lean diagnostics from the candidate being repaired. The primary comparison is C versus B; C versus A and B versus A are secondary.

The repository is not yet at the evidence stage for answering this question. The current inventory has 96 candidates and zero eligible records; the test split is not frozen; the protocol remains proposed; and no paired API run manifest exists. We make no performance claim. Contributions at this stage are an experiment runner and analysis specification, a transparent candidate inventory, and a separation of compiler verification from semantic review.

Research questions.

1. Does C improve verified proof success over equally budgeted B on the same fixed Lean theorem and shared initial candidate?
2. How do A, B, and C compare at repair budgets 0, 1, 2, and 3?
3. Which first-attempt compiler-error categories are associated with repair success, and how uncertain are category-specific estimates?
4. How often does the complete natural-language-to-Lean path produce a statement independently judged faithful, separately from fixed-theorem repair?

2 Related Work

Benchmarks cover different points in formal mathematics. miniF2F provides formalized contest-level statements across proof assistants and reports neural proving baselines [10]. ProofNet pairs undergraduate-level natural-language theorem statements with Lean 3 statements and natural-language proofs; its human assessment of statement formalization underscores that type-checking alone is not the semantic target [1]. LeanDojo provides programmatic Lean interaction and a large library-derived theorem-proving benchmark, with retrieval-augmented proving as a central contribution [8].

Lean is an extensible theorem prover and programming language [4]; Mathlib supplies a large community-maintained library of formal mathematics [6]. These systems machine-check proofs but do not decide whether an informal statement was translated faithfully.

Compiler-conditioned repair is studied directly in APRIL, which pairs diagnostic-aligned Lean proof failures with repairs as supervised data [7]. Related work evaluates faithful natural-language-to-Lean translation and measures a gap between compilation and semantic acceptance [9]. Our planned study is a paired, inference-time comparison of feedback conditions on fixed undergraduate-level theorems. This is a design distinction, not a claim of priority or a measured advantage; current data support no comparative result.

3 System and Verification Safeguards

The end-to-end pipeline proposes a Lean formalization, drafts a proof sketch, synthesizes a proof, invokes Lean, and can repair failed candidates. The paired experiment runner is separate: it fixes the theorem, generates one initial candidate, and reuses it for A, B, and C. Traces preserve prompts, candidate and response text, compiler diagnostics, verification outcomes, model metadata, timestamps, and cost-reservation information when available.

The verifier requires a successful Lean process exit and applies the forbidden-shortcut policy (‘sorry‘, ‘sorryAx‘, ‘admit‘, unapproved declarations and axioms). This establishes acceptance for the supplied Lean proposition under the configured environment only. Independent human review is separately required for the English statement, theorem, assumptions, self-containment, family, split, and semantic fidelity. Compilation is not evidence of semantic equivalence.

4 Dataset Construction and Review

The draft has 21 baseline candidates. Nine textbook acquisition batches add 16, and a library-derived extension adds 59, for 96 total: 30 proposed development and 66 proposed test records. The inventory contains five categories and four difficulty levels. Sixteen candidates cite Oscar Levin's *Discrete Mathematics: An Open Introduction*; 21 baseline records have no item-level citation in their current records. Levin identifies a CC BY-SA 4.0 license, but repository rights review remains incomplete; source pages and the textbook PDF were archived on 27 October 2026 [2]. Public availability is not itself a redistribution rights determination.

The curation inventory records 96 strictly theorem-proof-verified candidates. Twenty-five original statements gained explicit exact-type proof checks; the formerly missing geometric-sum definition is now supplied in a hash-pinned candidate-context module. Expected-statement placeholders used for type comparison are never counted as target proofs. It records zero completed independent reviews and zero eligible records. Candidate records also have unresolved duplicate/family warnings. The target is at least 60 eligible held-out test examples and 15–20 separate eligible development examples, with each mathematical family in one split. Proposed split labels are not evaluation splits until review and freeze requirements pass.

Table 1 is generated from saved candidate records. It shows the inventory only, not the composition of an eligible or frozen dataset.

Table 1. Current candidate inventory by category, difficulty, provenance, and proposed split. Counts are candidates, not eligible examples.

Proposed split	Category	Candidates	Difficulty 1/2/3/4	Cited	No item citation	Eligible
development	arithmetic_identity	6	4/1/1/0	1	5	0
development	divisibility	7	3/3/1/0	4	3	0
development	induction	9	0/1/7/1	5	4	0
development	inequality	5	2/3/0/0	2	3	0
development	modular_arithmetic	3	0/3/0/0	3	0	0
test	arithmetic_identity	12	0/12/0/0	12	0	0
test	divisibility	19	1/18/0/0	16	3	0
test	induction	9	0/9/0/0	9	0	0
test	inequality	14	0/14/0/0	14	0	0
test	modular_arithmetic	12	0/10/0/2	9	3	0
Total	All categories	96	–	75	21	0
Proposed split totals: development=30, test=66; eligible total=0.						

5 Experimental Protocol

The original Groq machine-readable protocol is version 0.2.0 and has status `proposed_locked_for_review_not_preregistered`. It proposes the mutable

Groq model alias `openai/gpt-oss-120b`; account generation access and an immutable model revision are unverified. Exact prompts are referenced by SHA-256 in the protocol. A makes one initial call and zero repairs. B and C share that initial candidate and allow at most three repairs each. B receives no compiler feedback; C receives diagnostics from the corresponding failed verification. The configured environment is Lean `leanprover/lean4:v4.32.0` with Mathlib pinned by `lean_project/lake-manifest.json`.

The B and C template files intentionally have identical bytes: B inserts a fixed no-diagnostics message into the feedback field, while C inserts the immediately preceding structured compiler diagnostics. Rendered prompts and their hashes distinguish conditions.

The protocol has not passed independent review or preregistration. Pricing and model access evidence are incomplete, so API mode is not authorized. A valid run must bind exact dataset, protocol, prompts, model metadata, and Lean environment hashes, and use the same theorem and initial candidate across conditions. The protocol file hash is `ced0e392772700181188bddf3b59e67a5794980a28a0deaa4e03f5110e280861`. It specifies temperature 0.2, a 2,000-token completion cap, low reasoning effort, and no generation seed. The initial prompt SHA-256 is `00ec7bc08fdf14274d021a308ebc27a51e17e46198b4dbc29002b958a1e04990`; the blind and compiler-guided template files each have SHA-256 `98f5f2ca17094874da1b89f5b4cfc4f7eff00753dc56ac8d8cb7161fdaa57745`.

6 Proposed Local Model Configuration

A new proposed protocol, version 0.3.0-local, retains the paired research question and three-repair budget while replacing paid/API inference with the cached MLX conversion of Qwen2.5-7B-Instruct [5, 3]. This substitution requires supervisor approval because the professor requested API-generated experiments; no such approval is recorded. The model snapshot is `c26a38f6a37d0a51b4e9a1eb3026530fa35d9fed`; exact weight, configuration, tokenizer and chat-template hashes are recorded in the model inventory. Quantization uses four bits and group size 64. The proposed backend is MLX-LM 0.32.2 with MLX 0.32.4 on an Apple M4 with 16 GiB unified memory. A neutral READY-prompt generation verified backend operation; it is neither a mathematical pilot nor evidence for repair performance.

The proposed settings are temperature 0.2, top-p 1, a 2,000-token completion cap, an 8,192-token input-plus-output context bound, and one primary base seed 2027. Each logical call uses a deterministic seed derived from that seed, problem/condition/attempt context and prompt hash, so stopping B cannot shift C's random stream. Each request has an independent conversation and KV cache; replaying a logical call is rejected. A single seed limits conclusions about sampling variation; three-seed replication is not claimed and would require an approved problem-clustered analysis amendment before test access. Interrupted generation streams are retained; no automatic inference retry or paid-provider fallback occurs. Real development runs remain disabled pending eligible reviews

and approval, and held-out runs additionally require a valid freeze and audited pilot.

7 Results and Statistical Analysis

7.1 Availability of study results

There is no frozen eligible test denominator and no paired A/B/C API run manifest. Existing pilot manifests contain requested model strings but omit execution mode and paired conditions; they cannot establish API provenance or serve as study results. Historical manually authored compiler-checking demonstrations are not model pass@1. Table 2, Figure 1, and Table 3 are unavailable. We compute no rate, paired difference, interval, significance test, token total, latency estimate, or error-conditioned effect.

Table 2. Paired A/B/C results. Unavailable: no eligible frozen test records or paired API runs.

Outcome	Denominator	Status
First-attempt verified success	Eligible problems with completed verification	Unavailable
A success at budget 0	Eligible frozen test problems	Unavailable
B success at budgets 0–3	Eligible frozen test problems	Unavailable
C success at budgets 0–3	Eligible frozen test problems	Unavailable
Paired C–B, C–A, B–A	Complete matched problem pairs	Unavailable
Repair calls by condition	Eligible problem and condition	Unavailable
Calls, tokens, latency, wall time	Eligible problems / observed calls	Unavailable

Figure unavailable: there is no frozen eligible test set or paired API run. No historical chart is substituted.

Fig. 1. Verified success against repair calls 0–3, to be generated by the paired analysis pipeline after dataset and run-artifact gates pass.

7.2 Compiler-error taxonomy

The planned analysis uses Wilson 95% intervals for proportions, problem-level paired bootstrap intervals for absolute differences, exact two-sided McNemar tests on complete matched pairs, and Holm correction across three planned comparisons. Missing outcomes remain identified in the full eligible-set denominator;

Table 3. First-attempt compiler-error taxonomy and repair outcomes. Unavailable for the paired study: no qualifying API runs.

Category	Paired problem denominator	Status
Syntax/parser, type/elaboration, identifier, tactic/goal, timeout, other	None	Unavailable

complete-case pairs are reported separately. Repeated seeds are not counted as independent problems. These are analysis rules, not findings. No statistical power claim is made.

7.3 Semantic fidelity and end-to-end success

All 96 candidate records lack an adjudicated semantic-fidelity label. There is no frozen test split or independent review result. Table 4 reports only the review state of the candidate pool; it is not a test-set faithfulness estimate. Fixed-theorem repair and end-to-end formalization are separate outcomes.

Table 4. Candidate semantic-review state, not test-set faithfulness results.

Review state	Candidate count
faithful	0
not faithful	0
uncertain	0
pending/unreviewed	96
Eligible frozen test records (no freeze)	0

7.4 Qualitative trace examples

The examples below are historical, manually authored compiler-checking trace slices. They show candidate changes and Lean outcomes, but have no authenticated API provenance and are not A/B/C experiments or model outputs. The trace files do not establish a complete autonomous repair trajectory or a final end-to-end semantic outcome. No generalization is inferred. **Evidence note.** These are ASCII-transliterated, truncated manual historical trace slices; exact UTF-8 text remains in the saved source artifacts; ‘success‘ below means only the saved attempt record’s compiler-verification outcome. No API provenance is claimed.

arith_001 (manual trace; attempts 1 and 2 both rejected).

Initial candidate excerpt:

```
import algebra.group_power
import algebra.ring

theorem add_sq_nat (a b : Nat) : (a + b) ^ 2 = a ^ 2 + ...
- Expand (a + b) squared using the binomial identity
```

Compiler message excerpt:

```
line 1: unknown module prefix 'algebra'

No directory 'algebra' or file 'algebra.olean' in the s...
/Users/sadiapeerzada/NLP-to-Formal-Language-Proof/lean_...
/Users/sadiapeerzada/NLP-to-Formal-Language-Proof/lean_...
/Users/sadiapeerzada/NLP-to-Formal-Language-Proof/lean_...
```

Attempt 1 to attempt 2 candidate diff:

```
-- attempt_1
+++ attempt_2
@@ -1,4 +1,3 @@
-import algebra.group_power
import algebra.ring

theorem add_sq_nat (a b : Nat) : (a + b) ^ 2 = a ^ 2 +...
```

Saved verifier outcomes: attempt 1 success=false; attempt 2 success=false. Exit codes are not recorded in this legacy trace.

arith_004 (manual trace; attempts 1 and 2 both rejected).

Initial candidate excerpt:

```
theorem sq_diff_plus_2ab (a b : Nat) (h : b <= a) : (a ...
have h1 : (a - b) ^ 2 = a ^ 2 - 2 * a * b + b ^ 2 := ...
rw [Nat.sub_def, Nat.mul_add, Nat.sq_diff]
have h2 : a ^ 2 - 2 * a * b + b ^ 2 + 2 * a * b = a ^...
ring
```

Compiler message excerpt:

```
line 5: Unknown constant `Nat.sub_def`
```

Attempt 1 to attempt 2 candidate diff:

```
-- attempt_1
+++ attempt_2
@@ -1,6 +1,6 @@
theorem sq_diff_plus_2ab (a b : Nat) (h : b <= a) : (a...
have h1 : (a - b) ^ 2 = a ^ 2 - 2 * a * b + b ^ 2 :=...
- rw [Nat.sub_def, Nat.mul_add, Nat.sq_diff]
+ rw [Nat.sq_diff]
have h2 : a ^ 2 - 2 * a * b + b ^ 2 + 2 * a * b = a ...
```

```
Saved verifier outcomes: attempt 1 success=false; attempt 2
success=false. Exit codes are not recorded in this legacy trace.
```

arith_003 (manual trace; attempts 1 and 2 both rejected).

Initial candidate excerpt:

```
theorem distributive_nat (a b c : Nat) : a * (b + c) = ...
begin
- Step 1: Expand the product using the distributive ...
rw mul_add,
- Step 2: No additional steps needed, the equation i...
```

```
Compiler message excerpt:
line 4: Unknown identifier ‘begin‘
```

```
Attempt 1 to attempt 2 candidate diff:
-- attempt_1
+++ attempt_2
@@ -1,12 +1,2 @@
-theorem distributive_nat (a b c : Nat) : a * (b + c) =...
-begin
- - Step 1: Expand the product using the distributive...
- rw mul_add,
- - Step 2: No additional steps needed, the equation ...
```

```
Saved verifier outcomes: attempt 1 success=false; attempt 2
success=false. Exit codes are not recorded in this legacy trace.
```

ineq_001 (manual trace; attempts 1 and 2 both rejected).

Initial candidate excerpt:

```
theorem at_most_n_sq (n : Nat) : n <= n ^ 2 := by
- Recall the definition of the square of a natural n...
have h1 : n ^ 2 = n * n := by
- Use the commutative property of multiplication t...
rw [mul_comm]
```

```
Compiler message excerpt:
line 5: unsolved goals
n : Nat
|- n ^ 2 = n * n
```

```
Attempt 1 to attempt 2 candidate diff:
-- attempt_1
+++ attempt_2
@@ -7,7 +7,7 @@
- Expand n * n using the distributive property of m...
have h2 : n * n = (1 + n) * n := by
- Since n is a natural number, it can be written ...
- rw [add_comm, add_self_add_right]
```

```
+ rw [add_comm, add_one]

Saved verifier outcomes: attempt 1 success=false; attempt 2
success=false. Exit codes are not recorded in this legacy trace.
```

mod_002 (manual trace; attempts 1 and 2 both rejected).

Initial candidate excerpt:

```
theorem sq_mod_four (n : Int) : n ^ 2 % 4 = 0 or n ^ 2 ...
begin
cases (Int.even_or_odd n) with k hk,
{
- n is even

Compiler message excerpt:
line 4: Unknown identifier ‘begin‘

Attempt 1 to attempt 2 candidate diff:
-- attempt_1
+++ attempt_2
@@ -1,22 +1,17 @@
+- The previous attempt failed due to incorrect syntax...
theorem sq_mod_four (n : Int) : n ^ 2 % 4 = 0 or n ^ 2...
-begin
- cases (Int.even_or_odd n) with k hk,
- {

Saved verifier outcomes: attempt 1 success=false; attempt 2
success=false. Exit codes are not recorded in this legacy trace.
```

8 Threats to Validity and Limitations

The main limitation is the absence of eligible data and paired API results. The candidate pool remains unreviewed, splits are not frozen, source rights and review evidence are incomplete. The historical Groq alias is mutable; the proposed local replacement is hash-pinned but has no approved mathematical pilot. The library-derived candidates may overlap pretraining data or expose named reference lemmas through Mathlib. These candidates require family and contamination review rather than automatic acceptance as independent tests. There are no repeated experimental trajectories, valid statistical comparisons, or evidence from which to assess power. Existing pilot manifests lack execution metadata needed to establish API provenance; historical proof demonstrations were manually authored.

Even successful Lean compilation establishes correctness only relative to a formal proposition and trusted environment, not faithfulness to English. Independent review may be limited by reviewer expertise and disagreement; reviewer provenance and adjudication must be recorded. No submission gate is claimed to have passed.

9 Reproducibility

The offline preparation command is `scripts/research_readiness.py--output-dir<fresh-directory>`, run with the repository Python environment. It records exact input hashes, combined candidate validation, failed freeze requirements and pending review packets. Existing study artifacts are preserved. The current environment records Python 3.13, optional Groq SDK 1.5.0, MLX-LM 0.32.2 and Lean 4.32.0; the package snapshot and verification logs are stored separately from model experiment traces. Timeouts terminate the compiler process group, and partial compiler output is retained. No local test or manual reference proof counts as an API outcome. The final completion report lists the actual checks and external dependencies. The manuscript remains incomplete pending genuine paired results, five authentic model repair examples and author metadata. The pre-results PDF was compiled using Tectonic 0.17.0 and the official Springer class 2.25; successful typesetting does not establish empirical readiness.

10 Conclusion

The repository defines a paired design for compiler-guided Lean proof repair, but the empirical question remains unanswered. Next steps are to acquire and review enough source-backed problems, freeze family-safe development/test splits, independently approve and preregister the protocol, verify pricing and model access, and then conduct an approved pilot. Results will be added only from validated raw artifacts and the frozen manifest.

References

1. Azerbayev, Z., Piotrowski, B., Schoelkopf, H., Ayers, E.W., Radev, D., Avigad, J.: ProofNet: Autoformalizing and formally proving undergraduate-level mathematics (2023), `https://arxiv.org/abs/2302.12433`
2. Levin, O.: Discrete mathematics: An open introduction (2021), `https://discrete.openmathbooks.org/dmoi3.html`, third edition, fifth printing. Source states CC BY-SA 4.0; repository rights review pending
3. Mlx lm: Large language model inference and fine-tuning for apple silicon. `https://github.com/ml-explore/mlx-lm`, installed version 0.32.2; accessed 27 October 2026
4. de Moura, L., Ullrich, S.: The Lean 4 theorem prover and programming language. In: Automated Deduction – CADE 28. Lecture Notes in Computer Science, vol. 12699, pp. 625–635. Springer (2021). https://doi.org/10.1007/978-3-030-79876-5_37
5. Mlx conversion of qwen2.5-7b-instruct, four-bit. `https://huggingface.co/mlx-community/Qwen2.5-7B-Instruct-4bit`, snapshot c26a38f6a37d0a51b4e9a1eb3026530fa35d9fed; model card and Apache-2.0 license evidence archived 27 October 2026
6. The mathlib Community: The Lean mathematical library. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs. pp. 367–381. ACM (2020). https://doi.org/10.1145/3372885.3373824

7. Wang, E., Chess, S., Lee, D., Ge, S., Mallavarapu, A., Alper, J., Ilin, V.: Learning to repair Lean proofs from compiler feedback. In: ICLR VerifAI Workshop (2026), https://arxiv.org/abs/2602.02990
8. Yang, K., Swope, A., Gu, A., Chalamala, R., Song, P., Yu, S., Godil, S., Prenger, R.J., Anandkumar, A.: LeanDojo: Theorem proving with retrieval-augmented language models. In: Advances in Neural Information Processing Systems. vol. 36 (2023), https://proceedings.neurips.cc/paper_files/paper/2023/hash/4441469427094f8873d0fecb0c4e1cee-Abstract-Datasets_and_Benchmarks.html
9. Zhang, K., Gallardo Candela, P., Murthy, S., Xie, Y., Wang, Z., Raissi, M.: Beyond compilation: Evaluating faithful natural-language-to-Lean statement formalization (2026), https://arxiv.org/abs/2606.31002
10. Zheng, K., Han, J.M., Polu, S.: minif2f: A cross-system benchmark for formal olympiad-level mathematics. In: International Conference on Learning Representations (2022), https://arxiv.org/abs/2109.00110